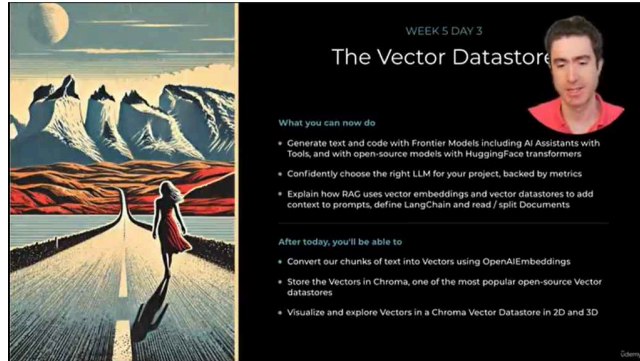


Day 3

01 Tháng Tư 2025 6:21 CH



Chuyển đổi văn bản thành vector embeddings:

- Bạn sẽ học cách chuyển đổi các chunks văn bản thành vector embeddings sử dụng OpenAI embeddings.
- OpenAI embeddings là một mô hình mã hóa (encoding model) được sử dụng để tạo ra các vector embeddings.

Lưu trữ vector embeddings trong Chroma:

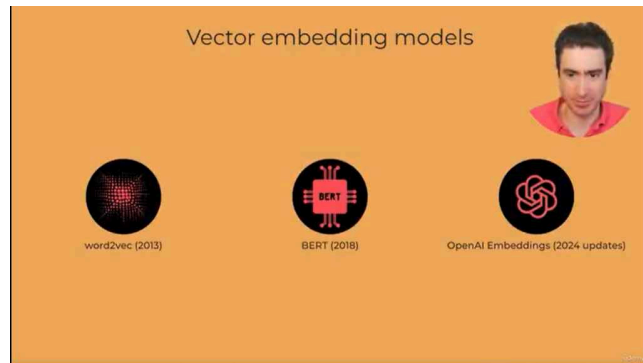
- Bạn sẽ học cách lưu trữ vector embeddings trong Chroma, một cơ sở dữ liệu vector mã nguồn mở phổ biến.

Trực quan hóa vector embeddings:

- Bạn sẽ học cách trực quan hóa các vector embeddings để hiểu rõ hơn về ý nghĩa của chúng.
- Bạn sẽ được khuyến khích thử nghiệm với việc tạo vector embeddings từ các văn bản khác nhau để củng cố sự hiểu biết.

Hiểu về mô hình tạo vector embeddings:

- Người nói sẽ giải thích về các loại mô hình khác nhau được sử dụng để chuyển đổi văn bản thành vector.



Phương pháp đơn giản để tạo vector embeddings:

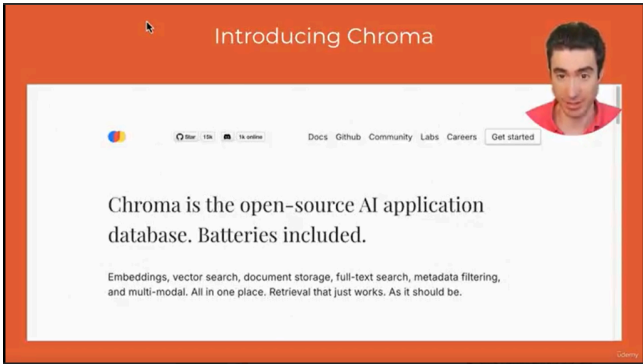
- Phương pháp này dựa trên việc đếm số lần xuất hiện của từng từ trong văn bản.
- Tạo một từ điển (vocabulary) chứa tất cả các từ có thể xuất hiện.
- Duyệt qua văn bản và đếm số lần xuất hiện của mỗi từ trong từ điển.
- Sử dụng số đếm này để tạo vector embeddings.
- Ví dụ: nếu từ "dog" xuất hiện 2 lần, giá trị 2 sẽ được đặt tại vị trí tương ứng với "dog" trong vector.
- Phương pháp này đơn giản nhưng không nắm bắt được ngữ cảnh và ý nghĩa của từ.

Các mô hình vector embeddings tiên tiến hơn:

- **Word2Vec (2013):**
 - Sử dụng mạng nơ-ron sâu (deep neural network) để chuyển đổi từ thành vector.
 - Nắm bắt được ý nghĩa của từ và mối quan hệ giữa các từ.
 - Ví dụ kinh điển: "king - man + woman = queen".
- **BERT (2018):**
 - Mô hình transformer do Google phát triển.
 - Nắm bắt được ngữ cảnh của từ trong câu.
 - Tạo ra vector embeddings chất lượng cao.
- **OpenAI Embeddings (2024 updates):**
 - Mô hình mới nhất từ OpenAI.
 - Được sử dụng trong khóa học này để tạo vector embeddings từ văn bản.
 - Được coi là mô hình tiên tiến nhất hiện nay.

Chroma:

- Một cơ sở dữ liệu vector mã nguồn mở phổ biến.
- Được sử dụng để lưu trữ và truy xuất vector embeddings.



Chroma là một cơ sở dữ liệu vector:

- Chroma là một ví dụ về cơ sở dữ liệu vector (vector data store).
- Nó được thiết kế đặc biệt để lưu trữ và truy xuất vector embeddings.

Các cơ sở dữ liệu khác cũng hỗ trợ vectors:

- Nhiều cơ sở dữ liệu chính thống hiện nay cũng hỗ trợ việc lưu trữ và tìm kiếm vector, ví dụ như MongoDB.

Chroma là một lựa chọn hàng đầu cho cơ sở dữ liệu vector:

- Chroma được xem là một trong những cơ sở dữ liệu vector hàng đầu.
- Nó được sử dụng để lưu trữ các vector embeddings được tạo ra từ dữ liệu văn bản.

Chức năng của Chroma:

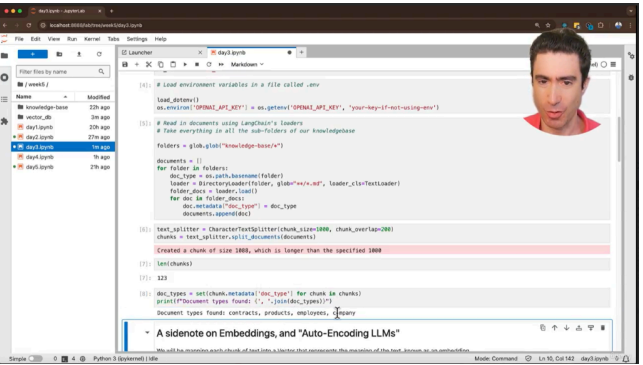
- Chroma cho phép thực hiện truy vấn trong ứng dụng AI và truy xuất dữ liệu từ các vector.
- Dữ liệu truy xuất được đưa vào prompt để truy vấn.

Sử dụng Chroma trong khóa học:

- Bạn sẽ sử dụng Chroma để lưu trữ vector embeddings trong khóa học này.

Bước tiếp theo:

- Bạn sẽ chuyển sang JupyterLab để thực hành sử dụng vector embeddings.



Xem vector embeddings trực tiếp:

- Bạn sẽ được xem các vector embeddings một cách trực quan.
- Điều này sẽ giúp bạn hiểu rõ hơn về ý nghĩa và cách chúng hoạt động.

Các thư viện cần thiết:

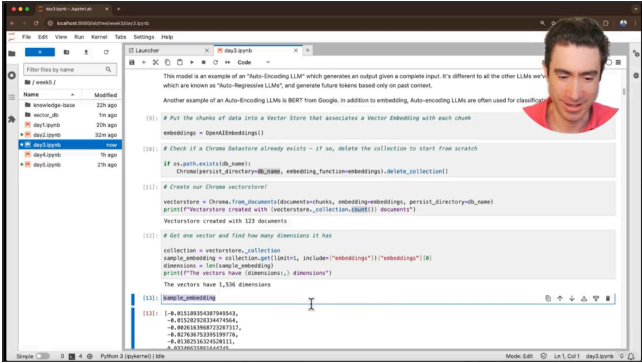
- Bạn sẽ sử dụng các thư viện từ LangChain để làm việc với OpenAI embeddings và Chroma.
- Bạn sẽ sử dụng t-SNE (t-distributed Stochastic Neighbor Embedding) để giảm chiều dữ liệu và trực quan hóa vector embeddings.
- Bạn sẽ sử dụng Plotly để tạo ra các biểu đồ trực quan.

Các bước thực hiện:

- Tải các biến môi trường (environment variables).
- Tải dữ liệu từ cơ sở tri thức (knowledge base) sử dụng LangChain loaders.
- Sử dụng text splitter để tạo ra các chunks (đoạn văn bản) từ dữ liệu đã tải.
- Tạo vector embeddings từ các chunks sử dụng OpenAI embeddings.
- Lưu trữ các vector embeddings trong Chroma.
- Trực quan hóa các vector embeddings sử dụng t-SNE và Plotly.

Mục tiêu:

- Hiểu rõ hơn về cách vector embeddings được tạo ra và sử dụng.
- Làm quen với các công cụ và thư viện cần thiết để làm việc với vector embeddings.
- Trực quan hóa dữ liệu vector để hiểu rõ hơn về mối quan hệ giữa các chunks văn bản.



Ôn lại về embeddings và auto-encoding LLMs:

- Nhắc lại sự khác biệt giữa auto-regressive LLMs (đã học trong khóa học) và auto-encoding LLMs.
- Auto-encoding LLMs lấy toàn bộ đầu vào và tạo ra một đầu ra duy nhất, trong trường hợp này là một vector.
- BERT là một ví dụ về auto-encoding LLM.
- OpenAI Embeddings được sử dụng trong bài học này.

Tạo Chroma vector datastore:

- Mã được thêm vào để xóa datastore cũ nếu nó tồn tại, tránh việc thêm dữ liệu trùng lặp.
- Datastore được lưu trữ trong thư mục "vector_db" (có thể tùy chỉnh).
- Chroma sử dụng SQLite để lưu trữ dữ liệu.

Tạo vector embeddings và lưu trữ trong Chroma:

- Sử dụng LangChain để tạo vector embeddings từ các chunks văn bản và lưu trữ chúng trong Chroma.
- Quá trình này được thực hiện chỉ trong một dòng mã.
- Số lượng documents (chunks) được lưu trữ trong datastore là 123, khớp với số lượng chunks đã tạo trước đó.

Kiểm tra kích thước của vector embeddings:

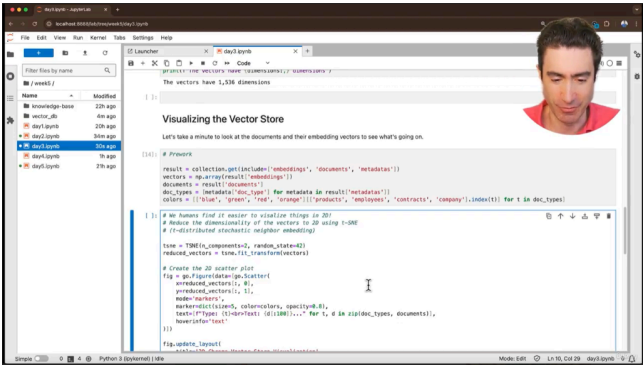
- Mỗi vector embedding có 1536 chiều (dimensions), nghĩa là 1536 số tạo thành một vector.
- Vector embeddings được in ra để xem cấu trúc của chúng.

Ý nghĩa của vector embeddings:

- Mặc dù các con số trong vector embeddings không có ý nghĩa trực tiếp đối với con người, chúng phản ánh ý nghĩa của chunk văn bản tương ứng.
- Các vector embeddings có tọa độ gần nhau trong không gian vector có ý nghĩa tương tự.

Trực quan hóa vector embeddings:

- Bước tiếp theo là trực quan hóa các vector embeddings để hiểu rõ hơn về cách chúng biểu diễn ý nghĩa của văn bản.



Trực quan hóa vector embeddings:

- Sử dụng t-SNE (t-distributed Stochastic Neighbor Embedding) để giảm chiều dữ liệu từ 1536 xuống 2D.
- Sử dụng Plotly để tạo biểu đồ phân tán (scatter plot) hiển thị các vector embeddings trong không gian 2D.
- Các điểm trên biểu đồ được tô màu khác nhau dựa trên loại tài liệu gốc (employees, contracts, products, about).
- Khi di chuột qua một điểm, sẽ hiển thị một đoạn văn bản ngắn từ chunk văn bản tương ứng.

Hiểu về không gian vector:

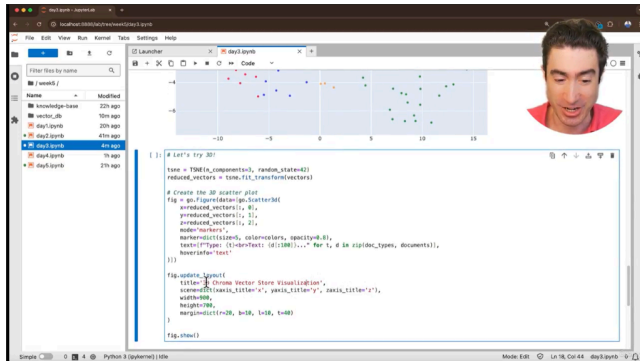
- Các chunks văn bản có ý nghĩa tương tự sẽ được nhóm lại gần nhau trong không gian vector.
- Các loại tài liệu khác nhau tạo thành các vùng riêng biệt trên biểu đồ.
- Các chunks từ hợp đồng mô tả tính năng sản phẩm được nhóm gần các chunks từ tài liệu sản phẩm, cho thấy sự tương đồng về ý nghĩa.
- Các chunks từ tài liệu "about" nằm ở trung tâm, cho thấy chúng chứa thông tin chung liên quan đến tất cả các loại tài liệu khác.

Thử nghiệm với vector embeddings:

- Bạn được khuyến khích thử nghiệm bằng cách thêm các chunks văn bản khác nhau vào cơ sở dữ liệu vector.
- Quan sát vị trí của các chunks mới trong không gian vector để hiểu rõ hơn về cách OpenAI Embeddings nắm bắt ý nghĩa của văn bản.

Trực quan hóa 3D:

- Bạn sẽ học cách tạo biểu đồ phân tán 3D để trực quan hóa vector embeddings.
- Việc chuyển đổi từ 2D sang 3D chỉ đơn giản là thay đổi tham số trong mã.



Trực quan hóa vector embeddings trong không gian 3D:

- Mã được điều chỉnh để tạo biểu đồ phân tán 3D thay vì 2D.
- Người nói nhận xét rằng biểu đồ 3D có thể không rõ ràng bằng biểu đồ 2D trong việc hiển thị sự phân tách giữa các loại tài liệu.
- Tuy nhiên, biểu đồ 3D cho phép tương tác và xoay để có cái nhìn tốt hơn về cách các vector embeddings được sắp xếp.

Tương tác với biểu đồ 3D:

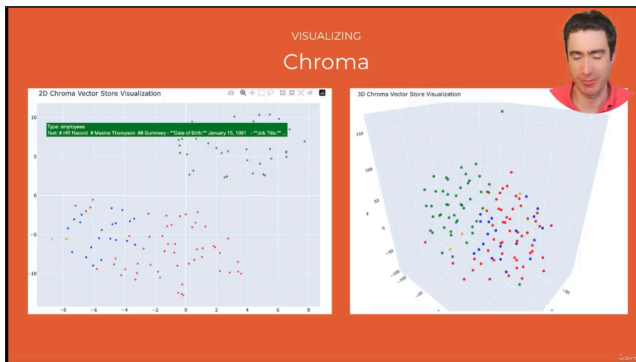
- Bạn có thể xoay và tương tác với biểu đồ 3D để khám phá không gian vector.
- Khi di chuột qua các điểm, bạn sẽ thấy nội dung văn bản tương ứng, tương tự như biểu đồ 2D.

Thử nghiệm với vector embeddings:

- Bạn được khuyến khích thử nghiệm bằng cách thêm các chunks văn bản của riêng mình vào cơ sở dữ liệu vector.
- Bạn có thể thêm tài liệu vào thư mục "knowledge base" hoặc sử dụng các lớp trình tải văn bản của LangChain.
- Bạn cũng có thể thay thế toàn bộ thư mục "knowledge base" bằng một thư mục mới chứa các tài liệu đơn giản để thử nghiệm.
- Mục tiêu là quan sát vị trí của các chunks văn bản mới trong không gian vector và hiểu rõ hơn về cách OpenAI Embeddings nắm bắt ý nghĩa của văn bản.

Chuẩn bị cho RAG (Retrieval-Augmented Generation):

- Việc thử nghiệm với vector embeddings sẽ giúp bạn hiểu rõ hơn về cơ sở của RAG.
- RAG là một kỹ thuật quan trọng sẽ được đề cập đến trong các phần tiếp theo của khóa học.



Thực hành với vector embeddings:

- Bạn được khuyến khích tự mình tạo các chunks văn bản và lưu trữ chúng trong một data store.
- Bạn cũng nên thử xem các vector embeddings trong cả không gian 2D và 3D để so sánh và lựa chọn.

Giới thiệu về Chroma:

- Chroma là một data store vector dễ sử dụng.
- Bạn đã được giới thiệu về cách sử dụng Chroma trong bài học trước.

Giới thiệu về FAISS:

- FAISS (Facebook AI Similarity Search) là một data store vector khác, cũng rất dễ sử dụng.
- FAISS là một data store vector trong bộ nhớ (in-memory).
- Việc chuyển đổi từ Chroma sang FAISS chỉ đòi hỏi thay đổi một vài dòng code.
- Bạn được khuyến khích thử nghiệm với FAISS.

Kết quả nhất quán:

- Nếu bạn sử dụng OpenAI embeddings, kết quả sẽ nhất quán giữa các data store vector khác nhau.

POPULATING THE VECTOR DATASTORE

This is where LangChain shines

Create the Chroma datastore and populate with Vector Embeddings of our Knowledge Base

..In 2 lines of code.

```
# Use OpenAI Embeddings model
embeddings = OpenAIEmbeddings()

# Create vectorstore
vectorstore = Chroma.from_documents(
    documents=chunks,
    embedding=embeddings,
    persist_directory=db_name
)
```



Ưu điểm của LangChain:

- LangChain cho phép thực hiện nhiều tác vụ phức tạp chỉ với một vài dòng code.
- Trong trường hợp này, việc tạo vector embeddings và lưu trữ chúng trong Chroma chỉ cần 2 dòng code.

Tạo embeddings bằng OpenAI embeddings:

- Dòng code `embeddings = OpenAIEmbeddings()` cho phép truy cập API của OpenAI để tính toán vector embeddings.

Tạo Chroma vector datastore:

- Dòng code `vectorstore = Chroma.from_documents(...)` tạo Chroma vector datastore từ các documents (trong trường hợp này là chunks văn bản).
- Hàm `from_documents` nhận vào 3 tham số:
 - `documents`: danh sách các documents (chunks) cần được chuyển đổi thành vector embeddings.
 - `embeddings`: đối tượng `OpenAIEmbeddings` được tạo ở bước trước.
 - `persist_directory`: tên thư mục để lưu trữ datastore (có thể tùy chỉnh).

Tạo embeddings cho toàn bộ documents:

- Thay vì tạo embeddings cho từng chunk, bạn có thể tạo embeddings cho toàn bộ documents.
- Thử thay thế `chunks` bằng `documents` trong dòng code `vectorstore = Chroma.from_documents(...)` để xem kết quả.
- Số lượng vector embeddings sẽ ít hơn nếu tạo cho toàn bộ documents.
- Bạn có thể so sánh kết quả trực quan hóa để xem sự phân tách giữa các documents.

PROGRESS

RAG time!

What you can now do

- Generate text and code with Frontier Models including AI Assistants with Tools, and with open-source models with HuggingFace transformers
- Confidently choose the right LLM for your project, backed by metrics
- Create and populate a Vector Datastore with the contents of a Knowledge Base

After the next session, you'll be able to

- Create a Conversation Chain in LangChain for a chat conversation with retrieval
- Ask questions and receive answers demonstrating expert knowledge
- Build a Knowledge Worker assistant with chat UI



Xây dựng RAG pipeline:

- Bạn đã sẵn sàng để xây dựng RAG (Retrieval-Augmented Generation) pipeline.
- RAG là một kỹ thuật kết hợp truy xuất thông tin và tạo văn bản để cải thiện chất lượng của các mô hình ngôn ngữ lớn (LLMs).

Sức mạnh của LangChain:

- LangChain cho phép kết hợp các thành phần khác nhau để tạo ra một giải pháp hoàn chỉnh chỉ với một vài dòng code.
- Điều này bao gồm việc tạo ra conversation chain (chuỗi hội thoại) và memory (bộ nhớ) để duy trì ngữ cảnh trong các cuộc trò chuyện.

Hội thoại hỏi đáp với kiến thức chuyên môn:

- Bạn sẽ học cách xây dựng một hệ thống hỏi đáp có thể trả lời các câu hỏi dựa trên kiến thức chuyên môn.
- Hệ thống này sẽ sử dụng RAG để truy xuất thông tin liên quan từ một nguồn dữ liệu bên ngoài và sử dụng thông tin đó để tạo ra câu trả lời.

Mục tiêu:

- Xây dựng một hệ thống RAG hoạt động.
- Sử dụng LangChain để tạo conversation chain và memory.
- Tạo một hệ thống hỏi đáp có thể trả lời các câu hỏi chuyên môn.

